

Browser History Simulation

Tran Thanh Nguyen
QE200015
FPT University
Quy Nhon Campus

Le Do Hoang Phuc
QE200057
FPT University
Quy Nhon Campus

Dao Cao Duy
QE190089
FPT University
Quy Nhon Campus

Phan Hoang Nam
QE190114
FPT University
Quy Nhon Campus

Abstract—The management of browser history is a fundamental aspect of modern web browsers, dictating the efficiency of user navigation. This paper presents the comprehensive design, architectural modeling, and performance evaluation of a simulated Browser History Management System. Unlike conventional array-based approaches that suffer from costly resizing operations, our system utilizes a purely pointer-based Doubly Linked List data structure to achieve optimal $O(1)$ time complexity for core navigation commands such as visiting new pages, and stepping backward or forward. Furthermore, we implemented advanced features such as keyword-based title searching, persistent data loading from CSV files, and arbitrary node deletion. To rigorously validate our design, we constructed an automated `DataGenerator` module capable of simulating massive volumes of user traffic, coupled with an `ExperimentRunner` module to benchmark execution latency. Empirical evaluations demonstrate the structural superiority of our implementation, proving its scalability and robustness in managing dynamic memory under high-load scenarios.

Index Terms—Browser History, Data Structures, Doubly Linked List, Time Complexity, Algorithm Design, Java Simulation.

I. INTRODUCTION

In the contemporary digital era, web browsers act as the primary interface between users and the internet. A seamless user experience heavily relies on the browser’s ability to swiftly transition between recently visited Uniform Resource Locators (URLs). When users navigate through multiple pages, they implicitly generate a traversal path. The browser must retain this path to facilitate the ubiquitous “Back” and “Forward” functionalities. The primary objective of this research project is to bridge the gap between theoretical data structure algorithms—taught in the CSD201 curriculum—and real-world software engineering challenges. While naive implementations might employ built-in dynamically resizing arrays (such as `ArrayList` in Java), such structures suffer from $O(n)$ overhead during memory reallocation and deletion operations. To mitigate these inefficiencies, we adopted a dynamic node-based approach. In this study, we propose a system centered around a Doubly Linked List (DLL). This structure maintains a sequential history where each entry (Node) retains references to both its chronological predecessor and successor. Our system guarantees constant *time* complexity, $O(1)$, for primary operations while offering flexible traversal. The remainder of this paper is structured as follows. Section II details the theoretical framework and the rationale behind selecting the Doubly Linked List. Section III provides an in-depth look at

our algorithmic implementation, including edge cases such as history truncation. Section IV outlines our methodology for stress-testing the system using the `DataGenerator` and `ExperimentRunner` modules. Finally, Section V concludes our findings and discusses potential avenues for future enhancements.

II. SYSTEM ARCHITECTURE AND DATA STRUCTURE RATIONALE

A. Evaluation of Alternative Data Structures

Before finalizing our architecture, several data structures were evaluated:

- **Dynamic Arrays (`ArrayList`):** While arrays provide excellent $O(1)$ random access, modifying the history (e.g., truncating the forward history upon visiting a new link from a past state) requires deleting elements at the end of the array. In worst-case scenarios, array resizing forces $O(n)$ memory copying.
- **Dual Stacks:** The Dual-Stack approach (using a `backStack` and a `forwardStack`) is a popular paradigm. It enforces strict Last-In-First-Out (LIFO) behavior. However, accessing or deleting historical elements positioned deep within the stack (e.g., clearing a specific URL for privacy reasons) requires popping all elements, yielding an unacceptable $O(n)$ time complexity.

B. The Doubly Linked List (DLL) Advantage

We selected the Doubly Linked List due to its inherent flexibility. A DLL is composed of independently allocated Node objects, each containing:

- `String url`: The web address.
- `String title`: The title of the webpage.
- `LocalDateTime timestamp`: The exact temporal signature of the visit.
- `Node prev`: A pointer to the previously visited page.
- `Node next`: A pointer to the forward page.

The `BrowserHistory` class maintains three critical references: `head` (oldest entry), `tail` (newest entry), and `current` (the user’s current viewing state). By manipulating these pointers, we bypass the need for continuous memory reallocation.

III. IMPLEMENTATION DETAILS

A. Core Navigation: Visit, Back, and Forward

The most computationally frequent operations are navigating the history. When a user executes the `visit(url)` method, a new node is instantiated. The critical algorithmic constraint here is handling the "branching" phenomenon. If a user travels back several steps and clicks a new link, the entire existing forward history must be orphaned and handed over to the Garbage Collector. Our `truncateForward()` method handles this in $O(1)$ time by nullifying the next pointer of the current node and updating the tail.

Listing 1. Handling forward truncation during a new page visit.

```
\subparagraph{
public void visit(String url, String title, LocalDateTime timestamp) {
Node newNode = new Node(url, title, timestamp);
if (head == null) {
head = tail = current = newNode;
} else {
if (current != tail) {
truncateForward(); // O(1) branch
}
current.next = newNode;
newNode.prev = current;
tail = current = newNode;
}
size++;
forwardCount = 0;
}}
```

B. Advanced Functionalities

To elevate the simulator beyond basic requirements, we implemented advanced data retrieval features:

- **Search By Title:** The `searchByTitle(String keyword)` method traverses the linked list from the head to search for specific string matches (case-insensitive). This sequential search operates in $O(n)$ time complexity.
- **Arbitrary Deletion:** Users frequently need to remove specific elements from their history. The `removePage(int index)` method traverses to the i^{th} node and safely rewires the prev and next pointers of adjacent nodes to bypass the target node. Special care is taken if the deleted node happens to be the head, tail, or the current pointer.
- **Persistent Storage Loading:** Utilizing Java IO, the `loadFromCSV()` method allows the system to initialize massive datasets previously generated, bypassing the need for manual data entry during debugging phases.

IV. PERFORMANCE EVALUATION AND BENCHMARKING

A. Experimental Setup

To empirically validate the time complexity bounds of our implementation, we developed two supplementary modules: `DataGenerator.java` and

`ExperimentRunner.java`. The `DataGenerator` synthesizes tens of thousands of randomized URLs, titles, and timestamps, formatting them into CSV structures. The `ExperimentRunner` then injects this data into the `BrowserHistory` class while utilizing `System.nanoTime()` to record the latency of each batch operation. The testing environment consisted of a standard academic workstation.

B. Complexity Analysis and Discussion

The theoretical complexity of our implemented functions is summarized in Table I. During the execution of

THEORETICAL COMPLEXITY ANALYSIS			
Function	Time Comp.	Space Comp.	Mechanism
<code>visit(url)</code>	$O(1)$	$O(1)$	Pointer manipulation
<code>back()</code>	$O(1)$	$O(1)$	Pointer traversal
<code>forward()</code>	$O(1)$	$O(1)$	Pointer traversal
<code>truncateForward()</code>	$O(1)$	$O(1)$	Pointer detachment
<code>searchByTitle()</code>	$O(n)$	$O(k)$	Sequential scan
<code>removePage(i)</code>	$O(n)$	$O(1)$	Search & Rewire

the `ExperimentRunner`, we simulated 50,000 continuous `visit()` operations. The empirical data strictly reflected the theoretical $O(1)$ bound; the latency per operation remained constant and did not degrade as the size of the Doubly Linked List grew. This stands in stark contrast to dynamic arrays, which would exhibit periodic latency spikes during array resizing triggers. The `removePage(int index)` operation, as expected, exhibited an $O(n)$ time complexity due to the lack of random access in linked lists, necessitating a sequential node-by-node traversal to reach the target index.

V. CONCLUSION

This project successfully designed, implemented, and thoroughly evaluated a Browser History Simulator utilizing a Doubly Linked List. By deliberately avoiding pre-packaged data collections and building the pointer logic from the ground up, we gained profound insights into memory management and algorithmic optimization. Our empirical benchmarks confirm that the DLL architecture is highly suited for sequential navigation models, offering strict $O(1)$ latency for `visit`, `back`, and `forward` actions. The incorporation of branching logic (`truncateForward`), substring searching, and arbitrary node deletion makes this simulation highly reflective of real-world browser engines like Chromium or Firefox. Future iterations of this project could explore Hybrid Data Structures. For instance, combining a Hash Map with the Doubly Linked List (similar to an LRU Cache) could theoretically reduce the $O(n)$ search time to $O(1)$ for direct URL lookups. Additionally, implementing strict memory constraints (e.g., dropping the oldest node when the list size exceeds a maximum threshold) would further optimize the system for mobile devices with limited RAM.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] FPT University, *CSD201: Data Structures and Algorithms Course Materials*, 2024.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.
- [4] M. T. Goodrich and R. Tamassia, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.